KDB-Means: A Novel Density-Based Approach to the K-Means Clustering Algorithm

1st Grace T. Bai *
Thomas Jefferson High School for Science and Technology
gracetbai@gmail.com

2nd Aneesh Pabolu
Thomas Jefferson High School for Science and Technology
aneesh.pabolu@gmail.com

***Abstract*—The K-Means algorithm is an unsupervised clustering algorithm, widely used due to its simplicity and efficiency. However, one of its major flaws lies in its suboptimal initialization of cluster centroids. The random initialization of centroids utilized by the K-Means algorithm often may lead to a loss of efficiency through an increased amount of calculations and iterations required for centroid convergence, as well as drastically different clustering results between trials despite training on the same data. Thus, our algorithm, which we will refer to in this work as KDB-Means (K-Density-Based Means), mitigates this issue of inefficiency through poor centroid initialization by using density metrics for the deterministic selection of initial centroids. We find that the integration of density weighting to the K-Means algorithm through KDB-Means improves clustering and decreases overall convergence time with better centroid initialization.**

***Index Terms*—centroids, clustering, unsupervised learning, K-Means**

I. Introduction

In recent years, there has been a significant increase in interest around the field of cluster analysis, or the classification of data into similar groups (clusters) based on their similarities [1]. Cluster analysis has many applications, from customer interest analysis in marketing to medical diagnoses in healthcare to risk assessment in finance [2]. An especially popular cluster analysis algorithm is the K-Means algorithm [3], an algorithm widely known as one of the strongest unsupervised learning algorithms. Its strengths lie particularly in its time efficiency, its clear, guaranteed convergence where the algorithm will always finish, and its comprehensibility. Due to these strengths, K-Means can separate data into groups effectively, and the algorithm's simplicity allows for more widespread use.

However, the most inefficient part of the K-Means algorithm comes from the repeated trials and calculations necessary to move the centroids and effectively classify the data into clusters. At the beginning of the K-Means algorithm, k centroids are randomly selected, often from the existing data points. As the algorithm progresses, these centroids will move in each iteration of the algorithm to minimize the total squared distances between each point and its nearest cluster centroid [2]. When the algorithm concludes, we are left with k clusters of data. If the number of centroid-moving trials can be reduced, we can significantly improve the efficiency of this algorithm. In fact, the time complexity of the original K-Means algorithm is

$$O(k \times n \times t), \tag{1}$$

where t is the number of iterations, k is the number of centroids, and n is the number of instances, or data points [10]. However, from this formula we can also see that the algorithm becomes much less efficient with an increasing dataset size. Thus, in this paper, we introduce a novel variation on K-Means to significantly minimize this weakness.

II. Related Work

In past years, there have been many improvements to the original K-Means algorithm as well as significant advancements in alternative clustering methods. The most notable improvement to the original algorithm is K-Means++ [4]. The most notable improvement on the K-Means algorithm with K-Means++ is the usage of distance for the spatial separation of initial centroids in the feature space. K-Means++ helps to mitigate the issue of picking weak initial centroids by initializing far-spread centroids through a probabilistic distance-based seeding strategy. However, while K-Means++ does improve centroid initialization by ensuring centroids are well-separated, it remains unaware of local data density, rendering this algorithm susceptible to the selection of subpar initial centroids in sparse regions or in outliers.

Density-based clustering methods offer an alternative by explicitly modeling dense regions of the dataset. One such algorithm that explicitly takes local data density into account when initializing centroids is the KNN-DP (k-Nearest Neighbor Density Peaks) [5]. While effective at locating density peaks, the KNN-DP algorithm does not optimize the K-Means objective function and lacks refinement steps, limiting clustering accuracy.

DBSCAN [6], another density-based clustering method, is a popular algorithm that groups points based on neighborhood density and identifies noise without requiring a predefined number of clusters. Although DBSCAN performs well on datasets with arbitrary cluster shapes, it is highly sensitive to parameter selection and doesn't create centroid-based clusters, making it less suitable for applications requiring centroid refinement or datasets with high dimensionality.

Our algorithm, KDB-Means, aims to address the limitations of these algorithms by taking advantage of local data density

when initializing centroids while retaining iterative refinement through the K-Means objective.

III. Datasets

In this work. we use the User Knowledge Modeling (UNS) Dataset [7] from the UCI Machine Learning Repository for performance analysis. The dataset is about students' knowledge status on Electrical DC Machines. The original dataset consisted of 5 features and 403 instances, where each instance represented one student. Each feature (STG, SCG, STR, LPR, and PEG) has continuous values that span the scale from 0 to 1. The feature STG (Study Time for Goal Object) represents the degree of time a user spends studying the target subject. SCG (The degree of repetition number of user for goal object materials) measures the frequency of repetition with which the user studies the target subject. This is reflective of how often a student revisits and practices the subject material. STR (Study Time for Related Objects) represents the degree of time the user spends studying subjects related to Electrical DC Machines. LPR (Performance for Related Objects) represents how the user has been performing on exams in subjects related to Electrical DC Machines. PEG (Performance for Goal Object) measured the user's exam performance in the Electrical DC Machines subject. The class label, UNS, is an ordinal categorical variable with the following labels: very_low, Low, Middle, High. A student with a UNS of "very_low" means the student has very low knowledge on the target subject (Electrical DC Machines). A student has low knowledge on the target subject if their UNS is "low". A student has average knowledge on the target subject if their UNS is "Middle". A student has above average knowledge on the target subject if their UNS is "High".

The User Knowledge Modeling dataset originally came with a training dataset with 258 instances and a testing dataset with 145 instances. However, due to the unsupervised nature of our KDB-Means algorithm, we combined both the original training and testing UNS datasets into one single training dataset. Additionally, the PEG feature was removed completely from the dataset in order to prevent data leakage, as the exam score performance in the subject of Electrical DC Machines would consist of the same information as the class label rather than a predictor of student knowledge.

The original testing dataset contains inconsistencies in the capitalization of UNS class labels, where the value of "very_low" in the original training dataset was represented as "Very Low" in the original testing dataset. During pre-processing, these labels were standardized and encoded as an ordinal categorical variable with the following mapping: "very_low" = 0, "Very Low" = 0, "Low" = 1, "Middle" = 2 and "High" = 3. It is important to note that the UNS class label was not used during clustering, and that these pre-processing steps were taken only to make the class label easier to work with for evaluation.

Prior to clustering, all continuous features were normalized to prevent attributes with larger variances from dominating distance and density computations. The normalization method we used and its underlying purpose in our algorithm is discussed in detail in the Methodology section (Section 4).

In addition to this dataset, we also created a number of toy (example) datasets for visualization purposes. These datasets were created using Scikit-Learn's make_blobs function [8], where the dataset had a random number of instances between 100 and 1000 (inclusive) as well as a random number of clusters between 3 and 10 (inclusive).

IV. Methodology

In this paper, we propose KDB-Means, a novel variation on K-Means. The core idea of our algorithm is to bias centroid initialization towards points that are well separated and representative of high-density regions. This is achieved by combining a local density estimate with distance-based separation. Our algorithm works especially well on unevenly distributed data, where K-Means and K-Means++ often show weaknesses.

In order for each feature to be represented equally with our density-based approach, all features are normalized using z-score normalization with this formula:

$$Z = \frac{X - \mu}{\sigma}, \tag{2}$$

where Z represents the standardized data point, X represents the raw data point, μ represents the average of that feature across the entire dataset, and σ represents the standard deviation, or data spread, of that feature across the dataset. By performing z-score normalization on our data, all our features will have a mean value of 0 and a standard deviation of 1, making our density calculations more even by preventing features with different scales from being weighted differently.

Once the features are normalized with the z-score normalization method, the density radius r_0 is calculated for each node (instance). In order to do this, we first compute the measure of data spread across the entire dataset by finding the average distance between each pair of nodes. We then multiply this average distance value by a predefined value α = 0.2, the optimal value determined from trial-and-error from α = 0 to α = 1. Then, using a NearestNeighbors method imported from Scikit-Learn, the full dataset is stored to record neighbors for each node within the circle of radius r_0, and the indices of each node's neighbors are returned. The number of a node's neighbors in the circle of radius r_0 thus represents that node's density, ρ.

Once the densities for all nodes are calculated, we then select the node with the highest density value ρ as our first centroid. After we select the first centroid, we then compute a value v for each remaining node to determine the best placements for the following centroids using this centroid-selection formula:

$$v = \rho \times d, \tag{3}$$

where p is the calculated density value of that node, and d represents the Euclidean distance from that node to the

nearest centroid, where Euclidean distance is calculated with this classic formula

$$d = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}, \tag{4}$$

where p_i and q_i represent feature values for two different nodes, and n represents the number of features, thus taking the square root of the summed squared distance between two nodes for each feature.

Our centroid-selection formula is a variant on the K-Means++ formula, which calculates the probability $p(x)$ of a node x being picked as the next centroid with

$$p(x) = \frac{d^2(x)}{\sum_{x' \in \mathcal{N}} d^2(x')}, \tag{5}$$

where d similarly represents the Euclidean distance from that node x to the nearest centroid and $\mathcal{N}$ represents the set of all nodes in the dataset. However, unlike the K-Means++ algorithm, our centroid-selection algorithm is completely deterministic in order to eliminate the possibility of drastically different outcomes with different initializations. The next centroids are then picked by selecting the node with the max value from our formula, where the formula values are recalculated in each centroid selection to account for updated distances to the nearest centroids, until all k centroids are initialized. This method allows for the optimal selection of centroids, where initial positions are far from existing centroids but also in high-density areas, thus allowing for a reduced number of centroid-moving iterations for clustering convergence through the typical K-Means algorithm. After this initialization, the typical K-Means algorithm proceeds with these preselected centroids using the Euclidean formula for distance.

We then evaluate the performance of each algorithm after convergence using a few metrics, which are the Adjusted Rand Index (ARI) [9]:

$$ARI(U,V) = \frac{2(N_{00}N_{11} - N_{01}N_{10})}{(N_{00} + N_{01})(N + 01 + N_{11}) + (N_{00} + N_{10})(N_{10} + N_{11})}, \tag{6}$$

where the ARI between the partial clusterings U and V is calculated with N_{00}, the number of pairs in different clusters in U and V, N_{01}, the number of pairs in the same cluster in U but different in V, N_{11}, the number of pairs in the same cluster in U and V, and N_{10} and N_{01}, disagreement indicators, the Adjusted Mutual Information (AMI) [9],

$$AMI(U,V) = \frac{I(U,V) - E(I(U,V))}{\max(H(U), H(V)) - E(I(U,V))}, \tag{7}$$

where $I(U,V)$ represents the mutual information of U and V, E is the expected value, and H is entropy, and the Silhouette Score [10]:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}, \tag{8}$$

where $s(i)$, the Silhouette Score, is calculated with $a(i)$, the average distance between the node i and all the points in its own cluster, and $b(i)$, the distance between i and the next nearest centroid. The ARI and AMI are external validation formulas [10], where the actual labels of the dataset are used to calculate the goodness value, while the Silhouette Score evaluates the formed cluster quality in an unsupervised way without the dataset labels [10]. In addition to these metrics, we also record the number of K-Means iterations required for each algorithm (KDB-Means, K-Means, K-Means++) to converge. The ARI measures the similarity between the predicted values through the clustering algorithm and the true labels by checking whether they're in the same clusters and ranges from -1 (random) to 1 (perfect). The AMI similarly measures likeness between predicted values and labels, but measures mutual dependence through shared information instead and varies from 0 (independence between the predictions and labels) to 1 (high agreement) [9]. Lastly, the Silhouette Score varies from -1 (very bad cluster formation; nodes are closer to other centroids than their own) to $+1$ (very good cluster formation) [10].

V. Experimentation

The above procedure was completed for both the toy datasets for visualization and the larger User Knowledge dataset. The toy datasets were used mainly for visualization purposes with 2D cluster graphs for each algorithm, since the lower dimensionality meant significant computational power was saved as compared to the User Knowledge dataset.

A toy dataset was generated with the random parameters described above.

The methodology described above was performed, and each of the three algorithms (KDB-Means, K-Means, K-Means++) was executed with this dataset. The performance metrics (ARI, AMI, Silhouette Score, and number of iterations) were recorded for each algorithm.

Visualizations of the trials were recorded for presentation purposes, which included the visualization of the original cluster formation, initial centroid instantiation, and final centroid and cluster results for each algorithm, as well as a Silhouette plot comparing the cluster formation resulting from each algorithm.

With the User Knowledge dataset, the methodology described above was performed, and the performance metrics were recorded. Due to the higher-dimensional nature of the User Knowledge dataset, no visualizations were created, as the toy dataset visualizations provided much more understandable and communicable results.

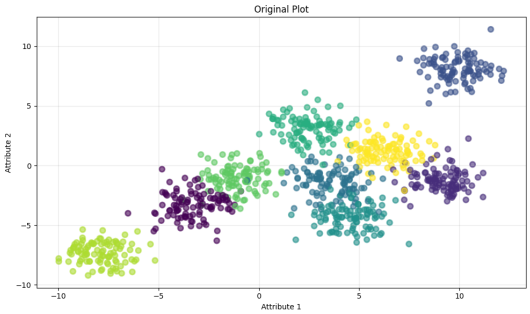

Fig. 1. Initial cluster formation in a 2D toy dataset with $N = 903$, $k = 9$

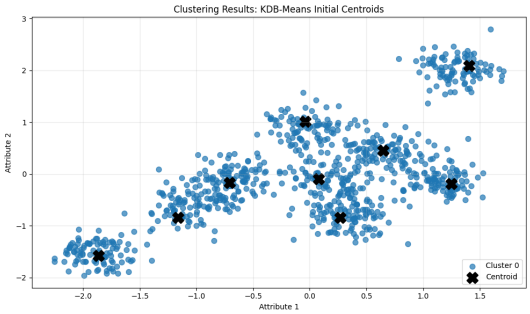

Fig. 2. KDB-Means initial centroid positions

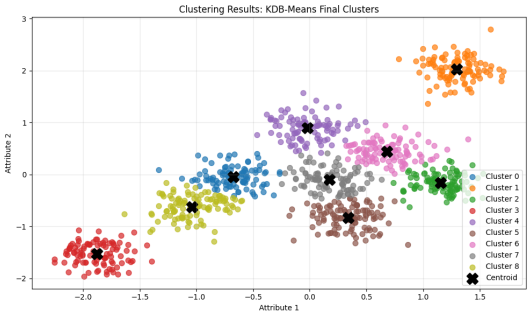

Fig. 3. KDB-Means final centroid positions with clusters

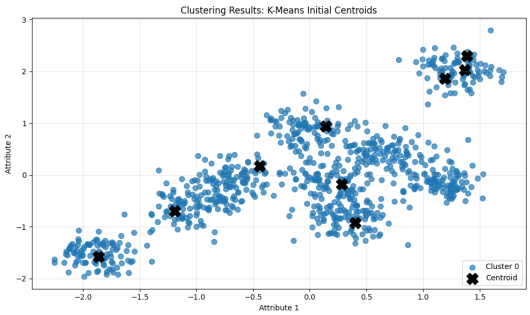

Fig. 4. K-Means initial centroid positions

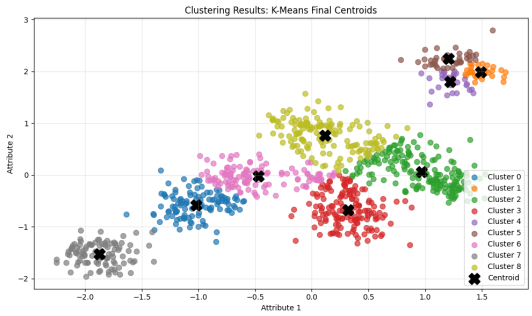

Fig. 5. K-Means final centroid positions with clusters

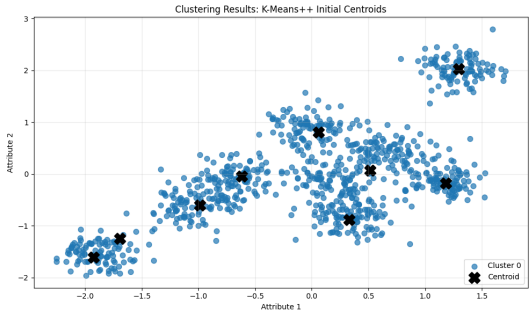

Fig. 6. K-Means++ initial centroid positions

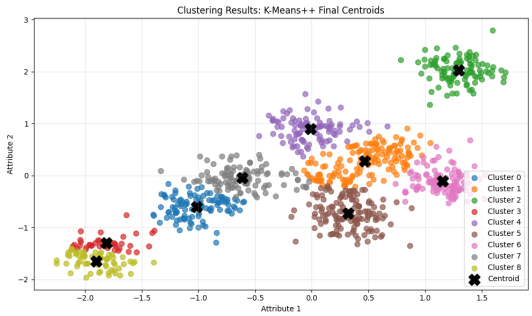

Fig. 7. K-Means++ final centroids positions with clusters

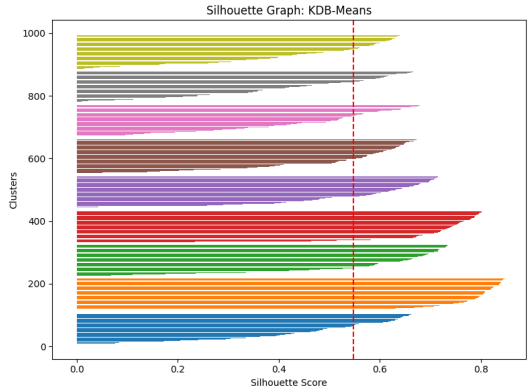

Fig. 8. Silhouette graph of the KDB-Means formed clusters

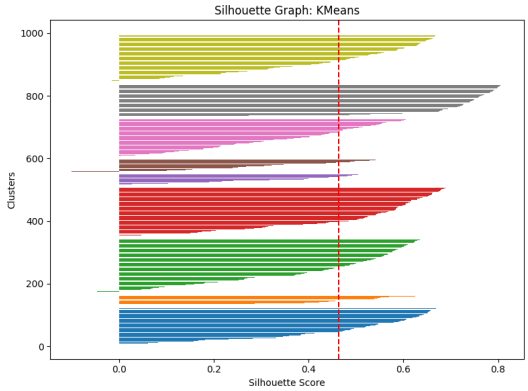

Fig. 9. Silhouette graph of K-Means formed clusters

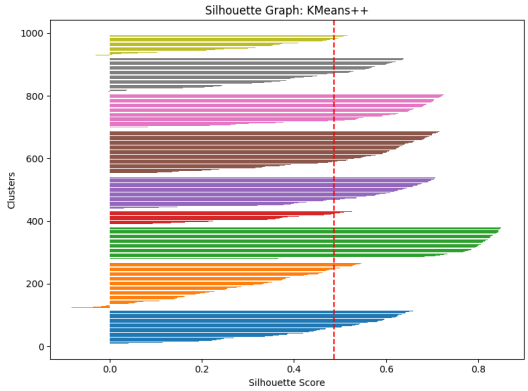

Fig. 10. Silhouette graph of K-Means++ formed clusters

VI. Results

As we can see from Figs. 2-7, the centroid initialization in KDB-Means from a toy dataset is significantly better than that of K-Means, and moderately better than that of K-Means++. Additionally, as we can see from Figs. 8-10, the silhouette scores of KDB-Means generally are better than that of K-Means and K-Means++, where the tall, uniform bars produced by the KDB-Means algorithm represent well-separated clusters.

From Table 1, we can observe that KDB-Means also consistently outperforms K-Means and K-Means++ from the AMI, ARI, Silhouette Score, and number of iterations. Averaged over 25 trials on the User Knowledge dataset, KDB-Means' AMI and ARI values are higher than that of K-Means and K-Means++, illustrating a strong agreement between the predicted clusters of KDB-Means and the actual labels. Additionally, the higher average Silhouette Score shows more well-formed and cohesive clusters.

VII. Conclusion

In this study, we have implemented a density-based variation on the traditional K-Means clustering algorithm through the integration of local density calculations with an improved K-Means++ centroid-selection procedure. We can conclude that

	KDB-Means	**K-Means**	**K-Means++**
AMI	0.2930	0.2225	0.2217
ARI	0.2108	0.1579	0.1556
Silhouette	0.1755	0.1703	0.1697
Iterations	12.0000	18.8400	16.2000

TABLE I
Clustering performance comparison, averaged across 25 trials.

the integration of density weighting to the K-Means algorithm through KDB-Means is effective and beneficial to the efficacy of the algorithm itself through better-formed clusters and decreased convergence time. By combining our novel density weighting with existing clustering algorithms, we develop a more efficient and accurate algorithm. It can be noted, however, that KDB-Means will work better in lower dimensions due to the nature of the density calculations within a n-dimensional circle of radius r_0, where an increased number of dimensions will work less well. KDB-Means does outperform the existing clustering methods of K-Means and K-Means++, however, in both toy and real-world datasets. Additionally, KDB-Means will likely work better with datasets of unequal density and will outperform other clustering algorithms due to its density-based nature, whereas typical clustering algorithms such as K-Means and K-Means++ would struggle to produce clusters due to their inherent density discrepancies. In the future, other applications can be considered with variants on K-Means or DBSCAN. In addition, the parameter can be tuned to better fit datasets of high-dimensionality to fix the weaknesses of KDB-Means surrounding datasets with many features. To conclude, this study has proved the usefulness and potential of the intersection between density-based algorithms and the classic K-Means clustering algorithm through the better overall performance of KDB-Means.

Acknowledgment

We are grateful to Brandon B. Le for assistance with the revisions in our paper and Dr. Yilmaz for support and guidance in this project.

References

[1] D. Henry, A. B. Dymnicki, N. Mohatt, J. Allen, and J. G. Kelly, "Clustering methods with qualitative data: A mixed-methods approach for prevention research with small samples," *Prevention Science*, vol. 16, no. 7, pp. 1007–1016, Oct. 2015.

[2] M. Shutaywi and N. N. Kachouie, "Silhouette analysis for performance evaluation in machine learning with applications to clustering," *Entropy*, vol. 23, no. 6, p. 759, 2021.

[3] J. A. Hartigan and M. A. Wong, "Algorithm AS 136: A K-means clustering algorithm," *Journal of the Royal Statistical Society, Series C*, vol. 28, no. 1, pp. 100–108, 1979.

[4] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," Stanford, Tech. Rep., 2006.

[5] J. Liao, X. Wu, Y. Wu, and J. Shu, "K-NNDP: K-means algorithm based on nearest neighbor density peak optimization and outlier removal," *Knowledge-Based Systems*, vol. 294, p. 111742, 2024.

[6] M. Ester, H.-P. Kriegel, J. Sander, X. Xu *et al.*, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, vol. 96, no. 34, 1996, pp. 226–231.

[7] "User knowledge modeling," UC Irvine Machine Learning Repository, 2013.

[8] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, "Scikit-learn: Machine learning in Python," *The Journal of Nachine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance," *Journal of Machine Learning Research*, vol. 11, no. 95, p. 2837–2854, 2010.

[10] P. J. Rousseeuw, "Silhouettes: a graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.